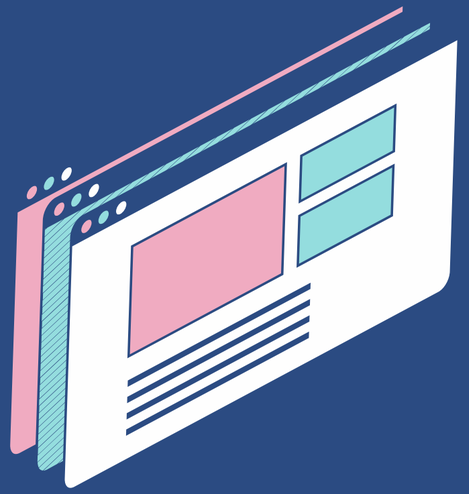
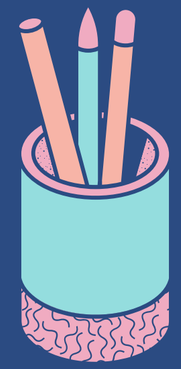


SLCHECK



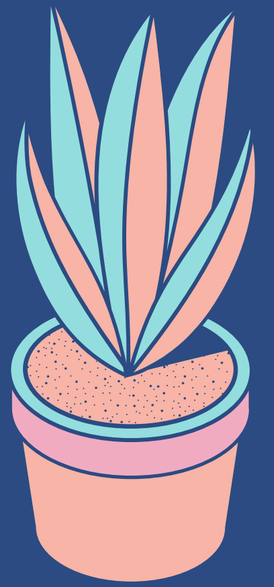


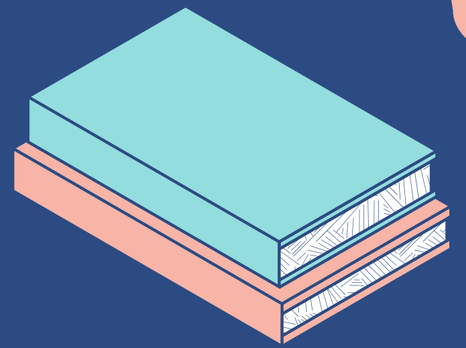
Apostu Cleopatra și Pricop Ania

UNIVERSITATEA DIN BUCUREȘTI
DEPARTAMENTUL DE INFORMATICA

INSTRUMENTE ȘI TEHNICI DE BAZA IN INFORMATICA

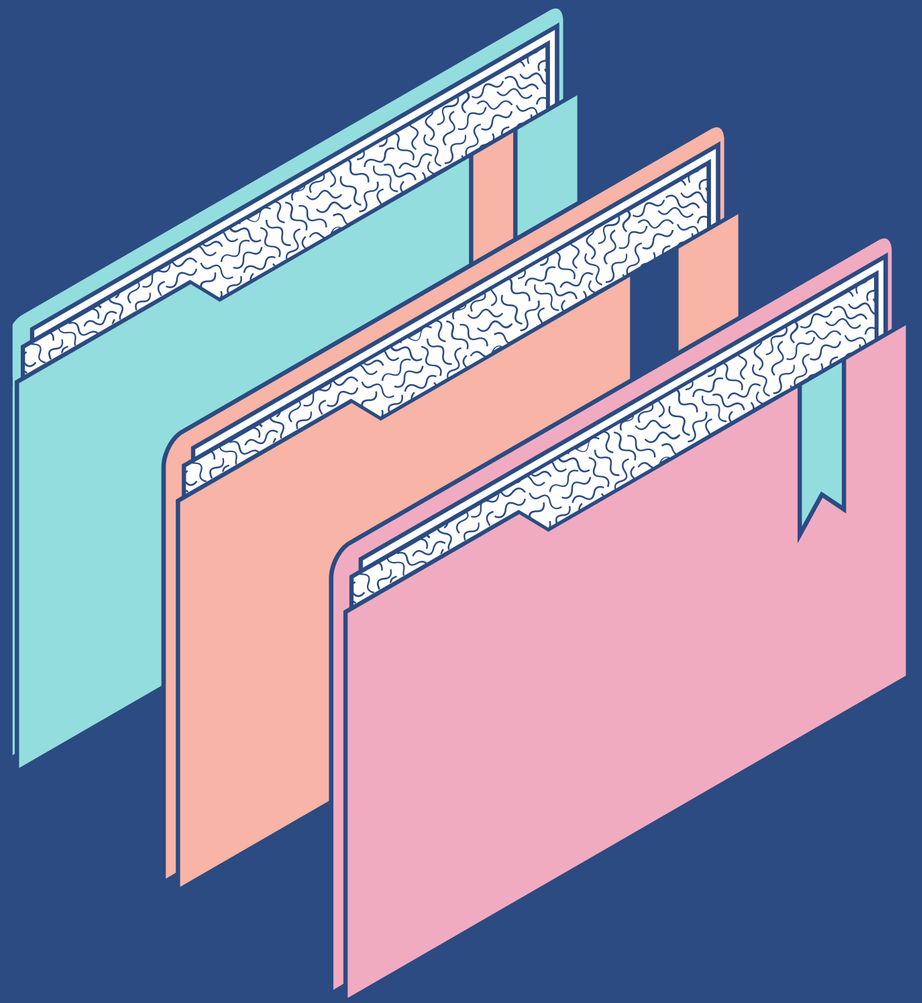
2025-2026





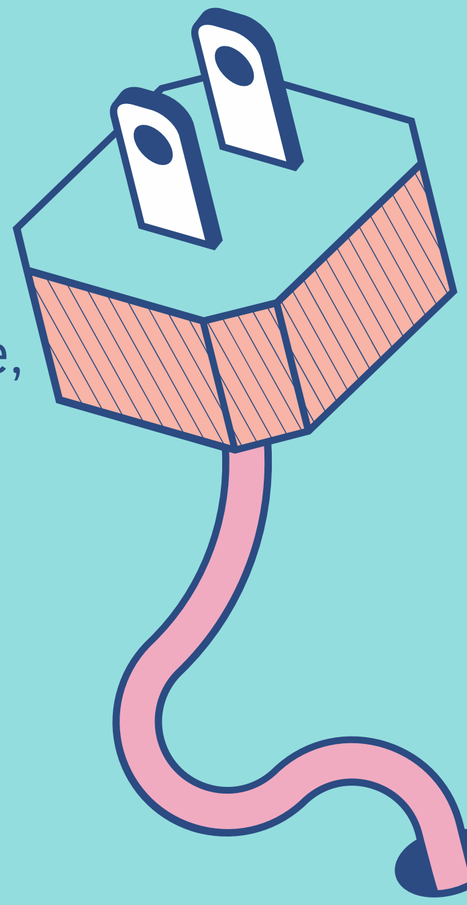
Outline

- Descrierea problemei
- Prezentarea soluției
- Design
- Implementare
- Experimente
- Concluzii



Descrierea problemei

- Administrarea sistemelor Linux implică gestionarea unor ierarhii complexe de fișiere.
- Link-urile simbolice(symlinks) sunt esențiale pentru flexibilitate, permițând referirea fișierelor din locații diferite.
- Problemă: Atunci când fișierul țintă este șters sau mutat, link-ul simbolic rămâne, dar devine invalid.
- Astfel sunt cauzate erori în aplicații, backup-uri incomplete sau confuzie pentru utilizatori.
- Identificarea manuală a acestor link-uri într-o structură adâncă de directoare este ineficientă și predispusă la erori umane.
- Din acest motiv, este necesar un instrument automatizat care să ”curețe” sau să raporteze aceste inconsistențe.



Specificația soluției

Obiective

- Crearea unui script Shell(SLcheck) care detectează link- uri simbolice invalide.
- Parcurgerea recursivă a unei ierarhii de directoare date ca parametru.

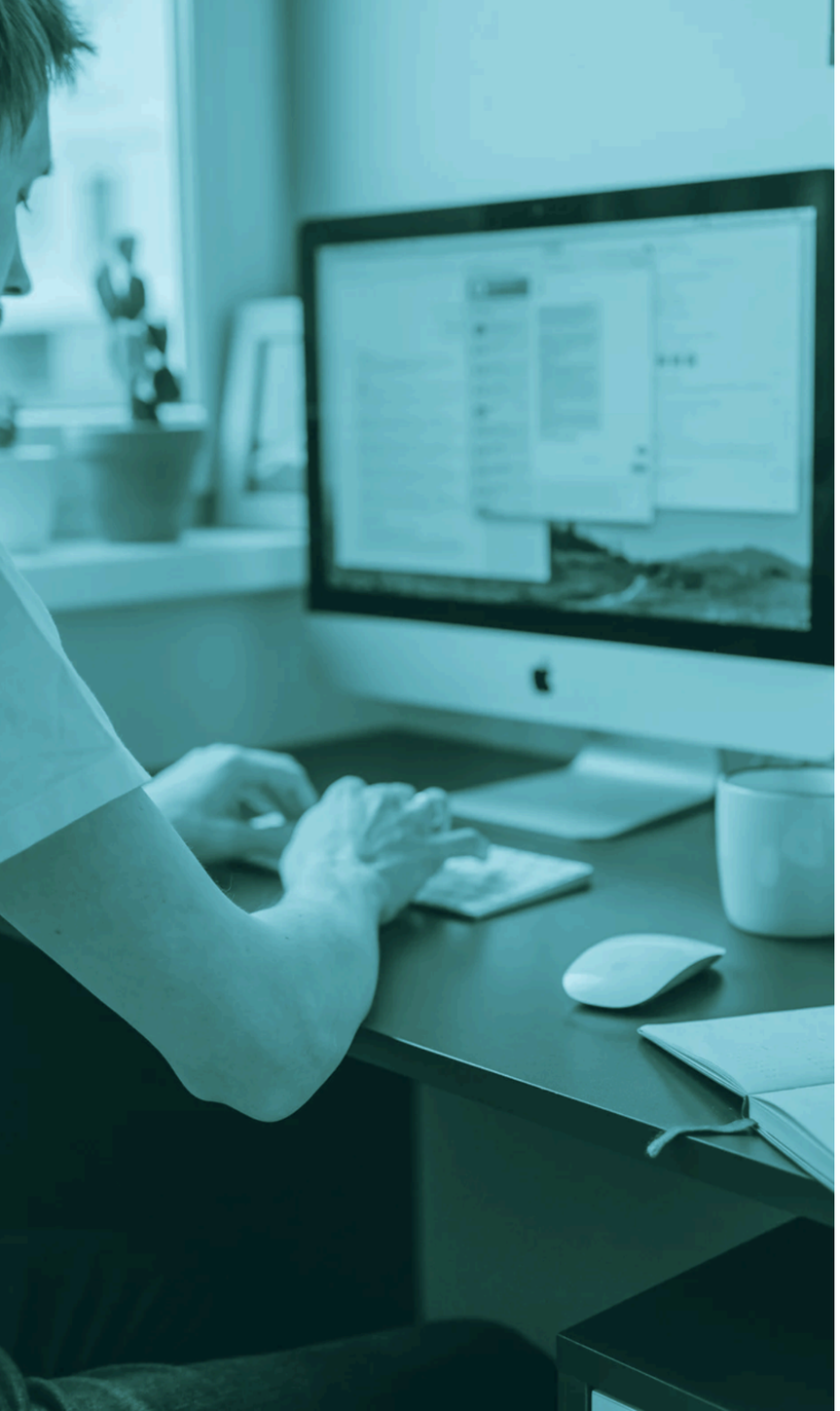
Cerințe funcționale

- Intrare: Calea către un director (ex: /home/user sau /)
- Detecție: Verificarea validității țintei pentru fiecare symlink înâlnit.
- Recursivitate controlată :
  - Implicit: Nu urmărește symlink-urile care duc către directoare(pentru a evita bucle infinite sau scanări redundante)
  - Flag --follow-symlinks : Permite explicit parcurgerea conținutului directoarelor referențiate prin symlink

---

Constrângeri

- Implementarea pură în Shell (Bash)
- Interzis : Utilizarea comenzii find pentru recursivitate; logica trebuie implementată manual



Design

ARHITECTURĂ ȘI ALGORITM

S-a optat pentru o abordare bazată pe Depth-First Search (DFS) implementată printr-o funcție recursivă

PSEUDOCOD SIMPLIFICAT

1. Funcția tranverse(directory)
2. Listează conținutul directory.
3. Pentru fiecare element :

- Dacă este Symlink: Verifică existența țintei . Dacă ținta lipsește → Raportează “Broken”
- Dacă este Director: Apelează tranverse(subdir).
- Dacă este Symlink către Director iar flag-ul --follow- symlinks este activ → Apeleaza tranverse(target), altfel → Ignoră

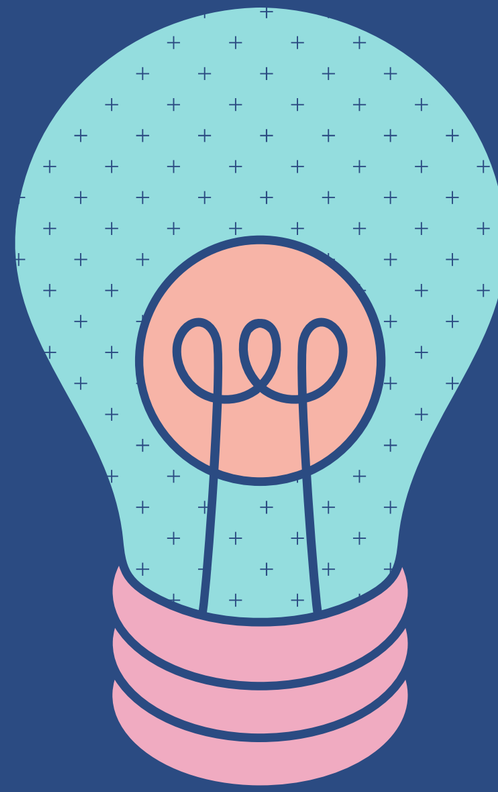
DE CE ACEASTĂ ABORDARE?

1. Oferă control asupra fișierului procesat
2. Permite implementarea logicii conditionale pentru flag-ul --follow-symlinks direct în fluxul de execuție, lucru mai dificil de realizat cu un simplu pipeline de comenzi

Implementare

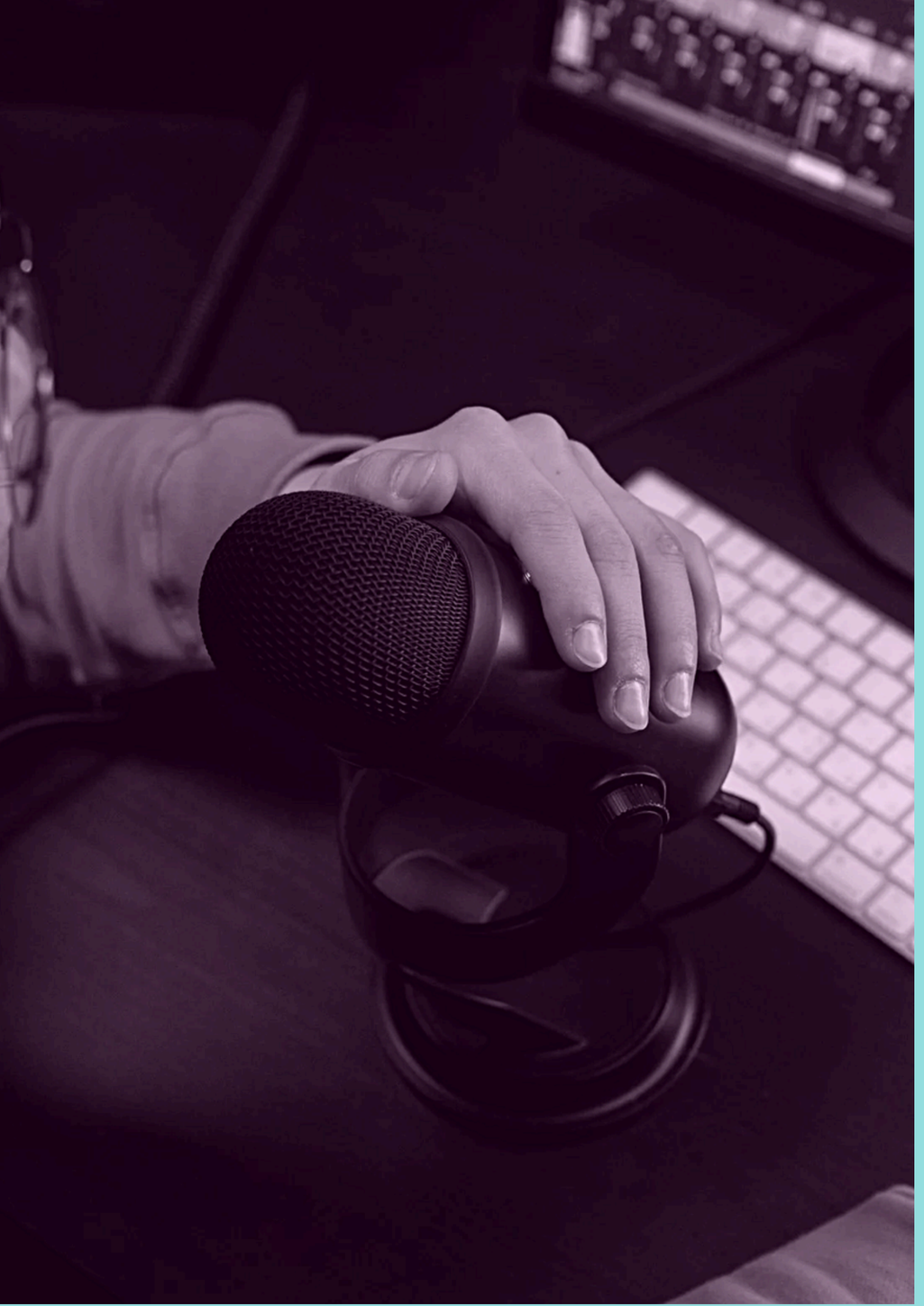
Componente Software:

- Limbaj: Bash Scripting.
- Primitive folosite: ls, test (sau []) , structuri de control (if, for, case).



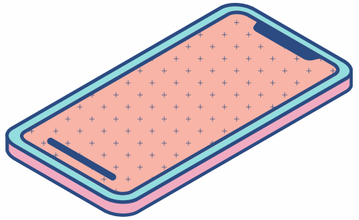
Detalii tehnice și provocări:

- Verificarean link-urilor: S-a folosit testul -L pentru a identifica un Symlink și -e pentru a verifica existența țintei.
- Formula if [-L “$file”] && [! -e “$file”]; then echo “Broken link: $file” ; fi
- Recursivitate manuală : Implementată printr-o funcție care se auto-apelează.



- Gestionarea numelor cu spații : S-a rezolvat prin utilizarea corectă a ghilimelelor și iterarea peste rezultatul globbing-ului (*) în loc de output-ul ls.

- Problema: Prevenirea buclelor infinite: În cazul folosirii --follow-symlinks, riscul de a parcurge o structura circulară este gestionat prin limitarea adâncimii





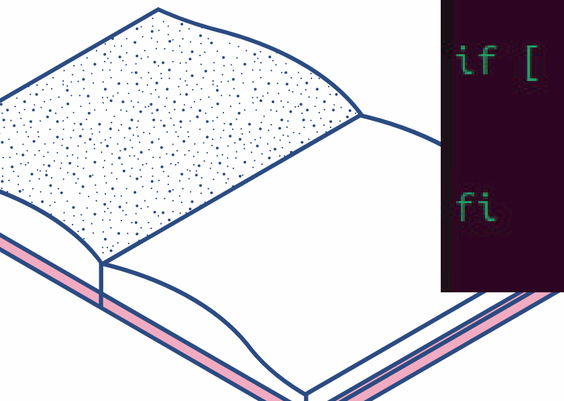
```
  GNU nano 7.2
#!/bin/bash

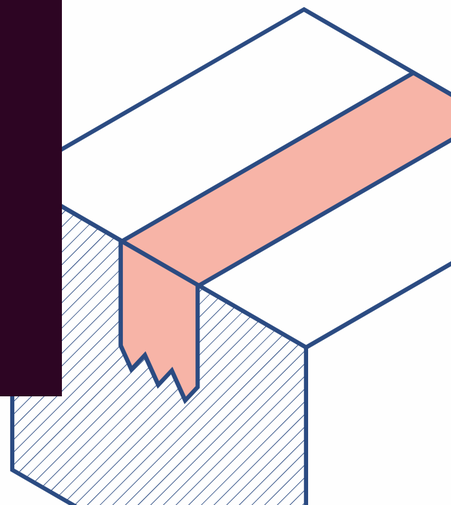
FOLLOW_SYMLINKS=false
TARGET_DIR=""

for arg in "$@"; do
        if [ "$arg" == "--follow-symlinks" ]
                then
                        FOLLOW_SYMLINKS=true
        elif [ -z "$TARGET_DIR" ]; then
                TARGET_DIR="$arg"
        fi
done

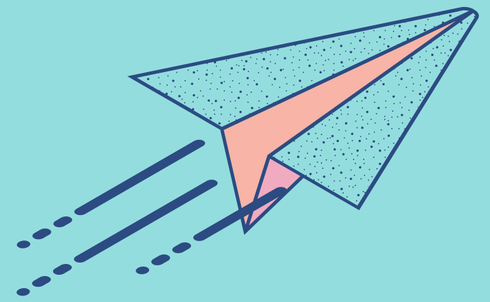
if [ -z "$TARGET_DIR" ]; then
        echo " Mod de utilizare: $0 [director] [--follow-symlinks]"
        exit 1
fi

if [ ! -d "$TARGET_DIR" ]; then
        echo "Eroare: Directorul '$TARGET_DIR' nu exista"
        exit 1
fi
```



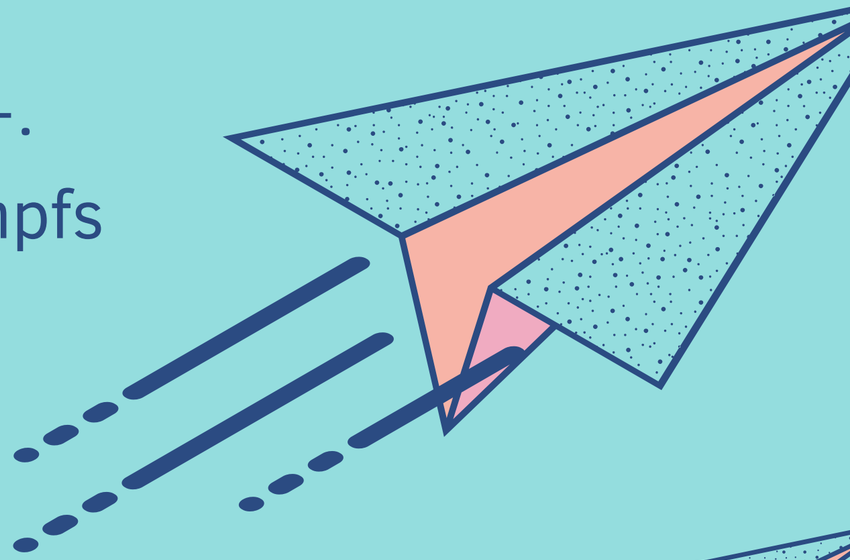


Experimente



Metodologie și testare:

- Mediu: Mașină virtuală Linux (Ubuntu) , Bash 5.0+.
- Scenariu de testare (Best practice) : Utilizarea tmpfs (RAM disk) pentru a simula link-urile rupte rapid.



Experimente Efectuate:

1. Validarea Recursivă a Structurii de Fișiere
2. Testare Avansată pe Sistem Volatil (tmpfs)



```
cleo@Ubuntu:~$ mkdir -p test/a test/b test/c
cleo@Ubuntu:~$ touch test/file1 test/file2
cleo@Ubuntu:~$ ln -s ../file1 test/a/link_to_file1
cleo@Ubuntu:~$ ln -s missing_file test/b/broken_link
cleo@Ubuntu:~$ ln -s ../b test/c/link_to_b
cleo@Ubuntu:~$ ln -s ../c test/b/link_to_c
cleo@Ubuntu:~$ mkdir test/protected
cleo@Ubuntu:~$ chmod 000 test/protected
cleo@Ubuntu:~$ ./slcheck.sh test
Analizez directorul: test
[BROKEN LINK] test/b/broken_link
Avertisment: Nu pot accesa directorul test/protected
cleo@Ubuntu:~$ ./slcheck.sh --follow-symlinks test
Analizez directorul: test
[BROKEN LINK] test/b/broken_link
[BROKEN LINK] test/b/link_to_c/link_to_b/broken_link
 Buclă detectată (inode deja vizitat): test/b/link_to_c/link_to_b/link_to_c
 Buclă detectată (inode deja vizitat): test/c/link_to_b
Avertisment: Nu pot accesa directorul test/protected
cleo@Ubuntu:~$ ./slcheck.sh
Utilizare: ./slcheck.sh [--follow-symlinks] <director>
cleo@Ubuntu:~$ ./slcheck.sh testX
Eroare: testX nu este un director valid
cleo@Ubuntu:~$ tree -L 2 test
test
├── a
│   └── link_to_file1 -> ../file1
├── b
│   ├── broken_link -> missing_file
│   └── link_to_c -> ../c
├── c
│   └── link_to_b -> ../b
├── file1
├── file2
└── protected  [error opening dir]
```

Validarea Recursivă a Structurii de Fișiere

Testul validează parcurgerea corectă a ierarhiei de directoare, identificarea legăturilor simbolice invalide și funcționarea mecanismului de protecție împotriva ciclurilor în structura de fișiere.

Testare Avansată pe Sistem Volatil (tmpfs)

```
ania@pricania:~/Documents/SLCheck$ cd ~/Documents/SLCheck
ania@pricania:~/Documents/SLCheck$ mkdir -p /tmp/ramdisk
ania@pricania:~/Documents/SLCheck$ sudo mount -t tmpfs -o size=20M tmpfs /tmp/ramdisk
[sudo] password for ania:
ania@pricania:~/Documents/SLCheck$ mkdir -p /tmp/ramdisk/test/a
ania@pricania:~/Documents/SLCheck$ mkdir -p /tmp/ramdisk/test/b
ania@pricania:~/Documents/SLCheck$ mkdir -p /tmp/ramdisk/test/c
ania@pricania:~/Documents/SLCheck$ touch /tmp/ramdisk/test/file1
ania@pricania:~/Documents/SLCheck$ touch /tmp/ramdisk/test/file2
ania@pricania:~/Documents/SLCheck$ ln -s missing_file /tmp/ramdisk/test/b/broken_link
ania@pricania:~/Documents/SLCheck$ ln -s ../file1 /tmp/ramdisk/test/a/link_to_file1
ania@pricania:~/Documents/SLCheck$ ln -s ../b /tmp/ramdisk/test/c/link_to_b
ania@pricania:~/Documents/SLCheck$ ln -s ../c /tmp/ramdisk/test/b/link_to_c
ania@pricania:~/Documents/SLCheck$ mkdir /tmp/ramdisk/test/protected
ania@pricania:~/Documents/SLCheck$ sudo chmod 000 /tmp/ramdisk/test/protected
ania@pricania:~/Documents/SLCheck$ ./SLCheck.sh /tmp/ramdisk/test
Analizez directorul: /tmp/ramdisk/test
[BROKEN LINK] /tmp/ramdisk/test/b/broken_link
Avertisment: Nu pot accesa directorul /tmp/ramdisk/test/protected
ania@pricania:~/Documents/SLCheck$ ./SLCheck.sh --follow-symlinks /tmp/ramdisk/test
Analizez directorul: /tmp/ramdisk/test
[BROKEN LINK] /tmp/ramdisk/test/b/broken_link
[BROKEN LINK] /tmp/ramdisk/test/b/link_to_c/link_to_b/broken_link
[INFO] Ciclu detectat sau director deja vizitat: /tmp/ramdisk/test/b/link_to_c/link_to_b/link_to_c (Sarit)
[INFO] Ciclu detectat sau director deja vizitat: /tmp/ramdisk/test/c/link_to_b (Sarit)
Avertisment: Nu pot accesa directorul /tmp/ramdisk/test/protected
ania@pricania:~/Documents/SLCheck$ ln -s /tmp/ramdisk/test link_catre_ram_complex
ania@pricania:~/Documents/SLCheck$ sudo umount /tmp/ramdisk
ania@pricania:~/Documents/SLCheck$ ./SLCheck.sh link_catre_ram_complex
Eroare: link_catre_ram_complex nu este un director valid
```

Scenariul demonstrează gestionarea completă a erorilor: detectarea buclelor infinite, tratarea directoarelor fără permisiuni și identificarea link-urilor rupte cauzate de demontarea bruscă a sistemului de fișiere (umount).

Concluzii:



Realizări:

- Am implementat un utilitar funcțional de system administration, respectând constrângeri (fără find).
- Am aprofundat mecanismele de funcționare ale sistemului de fișiere (inode-ul, symlinks).

Ce am învățat:

- Diferența între un link fizic(hard link) și unul simbolic(symlink).
- Necesitatea tratării cazurilor particulare (nume de fișiere ciudate, permisiuni de citire lipsă).



Vă mulțumim!